

# Aplicações e Serviços Web 2025/2026

## TP6: Validação e Documentação da API

### 1. Instalação e Configuração

Para começar, precisamos de instalar as ferramentas de validação (Zod) e as de documentação (Swagger).

Nota: Para este guião irá necessitar de ter todas as ferramentas dos guiões anteriores já instaladas. Pode ser útil copiar os ficheiros package.json e package-lock.json e de seguida correr o npm install caso esteja a usar uma nova pasta para este guião, desta forma todos os package irão ser instalados.

1. De seguida deve executar os seguintes comandos no terminal para instalar as ferramentas:

```
npm install zod@^4.0.0 swagger-ui-express@^5.0.0 swagger-jsdoc@^6.0.0
npm install -D @types/swagger-ui-express @types/swagger-jsdoc
```

### 2. Validação com Zod

1. Vamos criar um schema para validar a criação de um utilizador segundo um conjunto de regras definidas. Crie a pasta src/schemas e o ficheiro userSchema.ts.
2. Defina um objeto Zod adicionado o seguinte pedaço de código no ficheiro criado para garantir que o nome completo tem pelo menos 3 caracteres e o email é válido:

```
import { z } from 'zod';
export const userCreateSchema = z.object({
  fullName: z.string().min(3, "Mínimo 3 caracteres"),
  email: z.email("Email inválido")
});
```

3. No userController.ts, atualize o createUserController com o seguinte pedaço de código:

```
import { userCreateSchema } from '../schemas/userSchema.js';
```

```
export const createUserController = async (req: Request, res: Response) => {
  const result = userCreateSchema.safeParse(req.body);
```

```
  if (!result.success) {
    return res
      .status(400)
      .json({
```

```

      errors: result.error.issues.map(
        (issue) => `${issue.path}: ${issue.message}`,
      ),
    });
    const newUser = await userService.createUser(result.data.fullName,
result.data.email);
    res.status(201).json(newUser);
  };

```

4. No terminal, execute o script para ligar o servidor:  
**npx tsx watch src/server.ts**
5. Realize um pedido de POST à API com um email inválido (ex: teste@com) ou um nome com apenas 1 letra. A API deve devolver um erro 400 detalhando o que falhou, em vez de tentar inserir na base de dados. Depois teste inserir um utilizador válido e verifique se o mesmo foi inserido correctamente.

### 3. Expor Documentação com Swagger

1. Comece por criar o ficheiro `src/lib/swagger.ts` para configurar os metadados do swagger, o seu ficheiro deverá ter um pedaço de código similar a:

```

import swaggerJSDoc from 'swagger-jsdoc';

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'Library API',
      version: '1.0.0',
      description: 'Documentação organizada por ficheiros YAML'
    },
  },
  apis: ['./src/docs/*.yaml'],
};

export const specs = swaggerJSDoc(options);

```

2. No `server.ts`, adicione o seguinte pedaço de código para ativar a rota com a documentação:
- ```

import swaggerUi from 'swagger-ui-express';
import { specs } from './lib/swagger.js';

```

```
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));
```

3. Vamos agora adicionar a documentação para a rota GET dos users, para isso crie a pasta `src/docs` e o ficheiro `users.yaml`. Adicione o seguinte pedaço de código ao ficheiro criado:

```
paths:
```

```
  /api/users:
```

```
    get:
```

```
      tags: [Users]
```

```
      summary: Retorna a lista de todos os utilizadores
```

```
      responses:
```

```
        200:
```

```
          description: Sucesso
```

Nota: A indentação deve ser mantida caso contrário a documentação não irá funcionar devidamente. Além disso, poderá ser necessário reiniciar o servidor para que sejam refletidas as alterações ao visualizar a rota da documentação.

4. No terminal, execute o script para ligar o servidor:  
**`npx tsx watch src/server.ts`**
5. Aceda no browser à sua API na rota `api-docs`. Deverá ver uma interface gráfica listando o endpoint de utilizadores, permitindo testá-lo diretamente no browser através do botão Try it out.

### 3. Exercícios

Vamos utilizar os recursos Users, Books, Authors do guião anterior para validação com Zod e documentação com Swagger nos exercícios seguintes.

#### Exercício 1

Crie um esquema Zod para validar que o parâmetro `:id` em na pesquisa de utilizadores é um número inteiro positivo. Se o utilizador enviar `/api/users/abc`, o Controller deve impedir a chamada ao Service e devolver erro 400

#### Exercício 2

Implemente a validação para a criação de autores. O campo `name` deve ser obrigatório, ter no máximo 50 caracteres e não pode ser uma string vazia. Aplique esta validação no `authorController`. Certifique-se de que a documentação Swagger reflete que este campo é obrigatório.

#### Exercício 3

Ao pesquisar um livro pelo seu nome (`/api/books/search`), utilize o Swagger para documentar o parâmetro de query `title`. O utilizador deve conseguir escrever o título do livro diretamente na interface do Swagger para realizar a pesquisa.

#### Exercício 4

Crie um componente no Swagger chamado `ErrorResponse` (deve ser criado um ficheiro `components.yaml`). Use-o para documentar que todas as falhas de validação retornam um status 400 com a mesma estrutura de JSON.

#### Exercício 5

Crie uma validação Zod para a rota de atualização de dados de um livro. Considere a rota (`/api/books/:id`) que permite a atualização de dados de um livro. Crie um esquema Zod para validar que o `title` é string e o `authorId` é um número inteiro positivo antes de consultar a base de dados. O Controller deve impedir a chamada ao Service e devolver o erro 400 em pedidos que não cumpram com a regra. Documente esta rota com o Swagger.

## 5. Tutoriais

Deve realizar os tutoriais de suporte abaixo e consultar a documentação oficial:

- Zod
  - [Documentação Oficial](#)
  - [Zod com express](#)
- Swagger
  - [Documentação Oficial](#)
  - [Swagger-ui-express](#)
  - [Swagger YAML](#)
  - [OpenAPI Specification](#)